AULA 4

ANÁLISE DE ALGORITMOS

Luis Sato

Algumas funções matemáticas são definidas por fórmulas fechadas como: $f(x) = x^2 - 3x + 8$

Outras são descritas por relações de recorrência.

Por exemplo,

$$n! = \begin{cases} n \cdot (n-1)! & , \text{ se } n > 1 \\ 1 & , \text{ se } n = 1 \end{cases}$$

RECORRÊNCIA

$$n! = \begin{cases} n \cdot (n-1)!, & \text{se } n > 1 \\ 1, & \text{se } n = 1 \end{cases}$$

$$n! = \prod_{i=1}^{n} i = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot n$$

RECURSÃO

```
FATORIAL(n):
    se n=1:
        retorne 1
    senão:
        retorne n*FATORIAL(n-1)
```

```
FATORIAL(n)
    r=1
    para i de 2 a n:
    L r = r*i
    retorne r
```

muitos algoritmos que veremos são recursivos

LISTA 4A – Q1@

Escreva uma relação de recorrência para

$$\binom{n}{k}$$

usando o triângulo de Pascal

TRIÂNGULO DE PASCAL

n								
0	1							
1	1	1						
2	1	2	1					
3	1	3	3	1				
4	1	4	6	4	1			
5	1	5	10	10	5	1		
6	1	6	15	20	15	6	1	
7	1	7	21	35	35	21	7	1
k	0	1	2	3	4	5	6	7

LISTA 4A – Q1ⓑ

Escreva uma função recursiva para $\binom{n}{k}$

TRIÂNGULO DE PASCAL

n \ k	0	1	2	3	4	5	6	7
0	1							
1	1	1						
2	1	2	1					
3	1	3	3	1				
4	1	4	6	4	1			
5	1	5	10	10	5	1		
6	1	6	15	20	15	6	1	
7	1	7	21	35	35	21	7	1

```
BINOMIAL(n, k):
    se [ ⓐ ]:
        return 1
    senão:
        return [ ⓑ ]
```

MÁXIMO DIVISOR COMUM (mdc)

- O mdc entre números a e b (inteiros ≥ 1) é o maior d que é divisor de a e b
- Existe uma recorrência bem conhecida para o mdc: para $a \leq b$

operador de $\boxed{\text{resto}}$

$$\boxed{mdc(a,b) = \begin{cases} a, & \text{se } b\%a = 0 \\ mdc(b\%a, a), & \text{caso contrário} \end{cases}}$$

```
MDC(a,b):  — assumindo a ≤ b
  se a % b = 0:
      retorne a
  senão:
      retorne MDC(b%a, a)
```

$b \% a \leq a$?

LISTA 4A – Q2

Quais são as chamadas feitas por MDC(138, 940)

Exponenciação por quadrados

Suponha que você queira calcular 2^n para algum $n \geq 0$

```
POTÊNCIA2 (n)
    se n=0:
        retorne 1
    senão:
        x ← POTÊNCIA (n-1)
        retorne 2 * x
```

$T(n) =$ número de multiplicações (não exatamente o tempo de execução)

$$\boxed{T(n) = \begin{cases} T(n-1) + 1, & \text{se } n \geq 1 \\ 0, & \text{se } n = 0 \end{cases}}$$

Este é um exemplo de uma recorrência, uma função definida a partir do valor da própria função para valores menores de n

$$\left.\begin{aligned} T(n) &= T(n-1) + 1 \\ &= T(n-2) + 2 \\ &= T(n-3) + 3 \\ &= \ldots = T(0) + n \end{aligned}\right\} \boxed{\text{método da expansão}}$$

$$= 0 + n = \boxed{\Theta(n)}$$

Vamos procurar uma solução mais eficiente.

Se n é par: $2^n = 2^{\lfloor n/2 \rfloor} \cdot 2^{\lfloor n/2 \rfloor}$

se n é ímpar: $2^n = 2^{\lfloor n/2 \rfloor} 2^{\lfloor n/2 \rfloor} \cdot 2$

```
POTÊNCIA2_V2(n)
    se n=0 então retorne 1
    x ← POTÊNCIA(⌊n/2⌋)
    se n é par retorne x*x
    senão retorne x*x*2
```

Vamos escrever uma recorrência:

$$T(n) = \begin{cases} T(\lfloor n/2 \rfloor) + \overbrace{2}^{\text{no pior caso}} & , \text{se } n \geq 1 \\ 0 & , \text{se } n = 0 \end{cases}$$

Podemos simplificar para:

$$T(n) = \begin{cases} T(n/2) + 2 & , \text{se } n \geq 1 \\ 0 & , \text{se } n = 0 \end{cases}$$

$$
\begin{aligned}
T(n) &= T(n/2) + 1 \cdot 2 \\
&= T(n/4) + 2 \cdot 2 \\
&= T(n/8) + 3 \cdot 2 \\
&= T\left(\frac{n}{2^k}\right) + k \cdot 2 \\
&= \underbrace{T(0)}_{0} + (\lg n) \cdot 2 = \boxed{\Theta(\lg n)}
\end{aligned}
$$

$$
\frac{n}{2^k} \leq 1 \Leftrightarrow n \leq 2^k \Leftrightarrow \lg n \leq \lg(2^k) = k \lg 2 = k
$$

Busca binária: uma técnica básica, mas poderosa

A: um vetor ordenado

5	7	11	15	23	27	32	35	37	42	43	63	67	82	99
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Suponha que queiramos saber se um número x aparece em A. (Exemplo: 27)

O algoritmo mais simples é a busca linear.

```
BUSCA_LINEAR(A, n, x)
    para i de 1 a n:
        se A[i] = x:
            retorne VERDADEIRO
    retorne FALSO
```

Claramente o tempo de execução é $O(n)$, no pior caso $\Theta(n)$.

Mas essa solução não usa o fato de que A está ordenado

Ideia: "Olhar" sempre p/o elemento do meio e "ir" para o lado certo.

$x = 27$

meio

5	7	11	15	23	27	32	35	37	42	43	63	67	82	99
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

ir para cá

$x = 27$

meio

5	7	11	15	23	27	32
1	2	3	4	5	6	7

ir para cá

$x = 27$

meio

23	27	32
5	6	7

encontramos x

apenas 3 checagens vs 6

```
BUSCA_BINÁRIA (A, x, inicio, fim):
  l ← fim - inicio + 1   } comprimento
    l ≤ 0 :
    retorne FALSO
  meio ← ⌊(inicio + fim) / 2⌋

  se A[meio] = x :  retorne VERDADEIRO
  se A[meio] < x :  retorne BUSCA-BINÁRIA(A, x, meio+1, fim)
  se A[meio] > x :  retorne BUSCA-BINÁRIA(A, x, inicio, meio-1)
```

$T(n) = \Theta$ (número de chamadas para BUSCA-BINÁRIA)

$$\left.\begin{array}{ll} l = n & \text{na } 1^{\underline{a}} \text{ chamada} \\ l \leq n/2 & \text{na } 2^{\underline{a}} \text{ chamada} \\ l \leq n/4 & \text{na } 3^{\underline{a}} \text{ chamada} \\ \vdots & \\ l \leq n/2^{k-1} & \text{na } k\text{-ésima chamada} \end{array}\right\} \text{pode parar em qualquer chamada}$$

$$\frac{n}{2^{k-1}} < 1 \Leftrightarrow n < 2^{k-1} \Leftrightarrow \lg n < \lg(2^{k-1})$$
$$\Leftrightarrow k - 1 > \lg n$$

$$T(n) = O(\lg n)$$

$$\begin{cases} T(n) = T(n/2) + c \\ T(1) = c \end{cases}$$

Recorrência simplificada

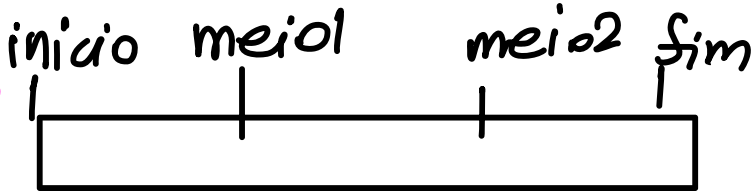
$$\begin{aligned} T(n) &= T(n/2) + c \\ &= T(n/4) + 2c \\ &= T(n/8) + 3c \\ &= T(n/2^k) + kc \quad = (\lg n) \cdot c \end{aligned}$$

$$\frac{n}{2^k} = 1 \Leftrightarrow n = 2^k \Leftrightarrow k = \lg n$$

$$T(n) = O(\lg n)$$

BUSCA_TERNÁRIA (A, x, inicio, fim):

$l \leftarrow$ fim − inicio + 1 } comprimento



se $l \leq 0$ retorne FALSO

$t \leftarrow \lfloor l/3 \rfloor$, meio1 ← inicio + t, meio2 ← inicio + 2t

se A[meio1] > x:

  retorne BUSCA_TERNÁRIA (A, x, inicio, meio1 − 1)

se A[meio1] = x retorne VERDADEIRO

se A[meio2] > x:

  retorne BUSCA TERNÁRIA (A, x, meio1 + 1, meio2 − 1)

se A[meio2] = x retorne VERDADEIRO

se A[meio2] < x:

  retorne BUSCA TERNÁRIA (A, x meio2 + 1, fim)

LISTA 4A – Q3

Simule para $x = 22$

3	6	7	8	9	11	15	17	20	22	31	42	45	50	53
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

LISTA 4A – Q4

Tempo de execução?

≥ 0

PROBLEMA: Para um vetor de inteiros A ordenado e um número s, decidir se $A[i] + A[j] = s$ para algum par (i, j) com $i \neq j$

```
BUSCA_SOMA_V1(A, s)
    para i de 1 a n-1:
        para j de i+1 a n:
            se A[i]+A[j]=s:
                retorne VERDADEIRO
    retorne FALSO
```

```
BUSCA_SOMA_V2(A,s)
  para i de 1 a n-1:
      f = s - A[i]
      se f ≥ 0:
          v ← BUSCA_BINÁRIA(A, f, i+1, n)
          se v = VERDADEIRO:
              return VERDADEIRO
  retorne FALSO
```

```
BUSCA_SOMA_V3(A,s):
    i ← 1
    j ← n
    enquanto i ≠ j:
        x = A[i]+A[j]
        se x = s:
            retorne VERDADEIRO
        senão se x > s:
            j ← j-1
        senão:
            i ← i+1
    retorne FALSO
```

LISTA 4A – Q5

Qual o tempo de execução de cada versão?
Qual versão é a mais eficiente?